

Name: V.Mariya Joevita

Reg.No: 3122235001077

Ex7: Bagging, Boosting, and Stacked Ensemble Models

```
pip install ucimlrepo
```

```
Collecting ucimlrepo
  Downloading ucimlrepo-0.0.7-py3-none-any.whl.metadata (5.5 kB)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2.2.2)
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2026.2.25)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.17.0)
Downloading ucimlrepo-0.0.7-py3-none-any.whl (8.0 kB)
Installing collected packages: ucimlrepo
Successfully installed ucimlrepo-0.0.7
```

1. Load and preprocess the dataset.

```
from ucimlrepo import fetch_ucirepo

# fetch dataset
breast_cancer_wisconsin_diagnostic = fetch_ucirepo(id=17)

# data (as pandas dataframes)
X = breast_cancer_wisconsin_diagnostic.data.features
y = breast_cancer_wisconsin_diagnostic.data.targets

# metadata
print(breast_cancer_wisconsin_diagnostic.metadata)

# variable information
print(breast_cancer_wisconsin_diagnostic.variables)
```

9	concave_points1	Feature	Continuous	None	None	None
10	symmetry1	Feature	Continuous	None	None	None
11	fractal_dimension1	Feature	Continuous	None	None	None
12	radius2	Feature	Continuous	None	None	None
13	texture2	Feature	Continuous	None	None	None
14	perimeter2	Feature	Continuous	None	None	None
15	area2	Feature	Continuous	None	None	None
16	smoothness2	Feature	Continuous	None	None	None
17	compactness2	Feature	Continuous	None	None	None
18	concavity2	Feature	Continuous	None	None	None
19	concave_points2	Feature	Continuous	None	None	None
20	symmetry2	Feature	Continuous	None	None	None
21	fractal_dimension2	Feature	Continuous	None	None	None
22	radius3	Feature	Continuous	None	None	None
23	texture3	Feature	Continuous	None	None	None
24	perimeter3	Feature	Continuous	None	None	None
25	area3	Feature	Continuous	None	None	None
26	smoothness3	Feature	Continuous	None	None	None
27	compactness3	Feature	Continuous	None	None	None
28	concavity3	Feature	Continuous	None	None	None
29	concave_points3	Feature	Continuous	None	None	None
30	symmetry3	Feature	Continuous	None	None	None
31	fractal_dimension3	Feature	Continuous	None	None	None

```
missing_values
0      no
1      no
2      no
3      no
4      no
5      no
6      no
7      no
```

```

19         no
20         no
21         no
22         no
23         no
24         no
25         no
26         no
27         no
28         no
29         no
30         no
31         no

```

```

X.columns
y.columns

```

```
Index(['Diagnosis'], dtype='object')
```

2. EDA

```

print(X.info())
print(y.info())
print(X.describe())

```

Data columns (total 1 columns):

```

#   Column      Non-Null Count  Dtype
---  -----  -
0   Diagnosis  569 non-null               object
dtypes: object(1)
memory usage: 4.6+ KB
None

```

```

count    radius1    texture1    perimeter1    area1    smoothness1  \
mean    14.127292    19.289649    91.969033    654.889104    0.096360
std      3.524049     4.301036     24.298981    351.914129    0.014064
min      6.981000     9.710000     43.790000    143.500000    0.052630
25%     11.700000     16.170000     75.170000    420.300000    0.086370
50%     13.370000     18.840000     86.240000    551.100000    0.095870
75%     15.780000     21.800000    104.100000    782.700000    0.105300
max     28.110000     39.280000    188.500000   2501.000000    0.163400

```

```

count    compactness1    concavity1    concave_points1    symmetry1  \
mean      0.104341      0.088799      0.048919      0.181162
std       0.052813      0.079720      0.038803      0.027414
min       0.019380      0.000000      0.000000      0.106000
25%       0.064920      0.029560      0.020310      0.161900
50%       0.092630      0.061540      0.033500      0.179200
75%       0.130400      0.130700      0.074000      0.195700
max       0.345400      0.426800      0.201200      0.304000

```

```

count    fractal_dimension1    ...    radius3    texture3    perimeter3  \
mean      0.062798    ...    16.269190    25.677223    107.261213
std       0.007060    ...     4.833242     6.146258    33.602542
min       0.049960    ...     7.930000    12.020000    50.410000
25%       0.057700    ...    13.010000    21.080000    84.110000
50%       0.061540    ...    14.970000    25.410000    97.660000
75%       0.066120    ...    18.790000    29.720000    125.400000
max       0.097440    ...    36.040000    49.540000    251.200000

```

```

count    area3    smoothness3    compactness3    concavity3    concave_points3  \
mean    880.583128    0.132369    0.254265    0.272188    0.114606
std    569.356993    0.022832    0.157336    0.208624    0.065732
min    185.200000    0.071170    0.027290    0.000000    0.000000
25%    515.300000    0.116600    0.147200    0.114500    0.064930
50%    686.500000    0.131300    0.211900    0.226700    0.099930
75%   1084.000000    0.146000    0.339100    0.382900    0.161400
max   4254.000000    0.222600    1.058000    1.252000    0.291000

```

```

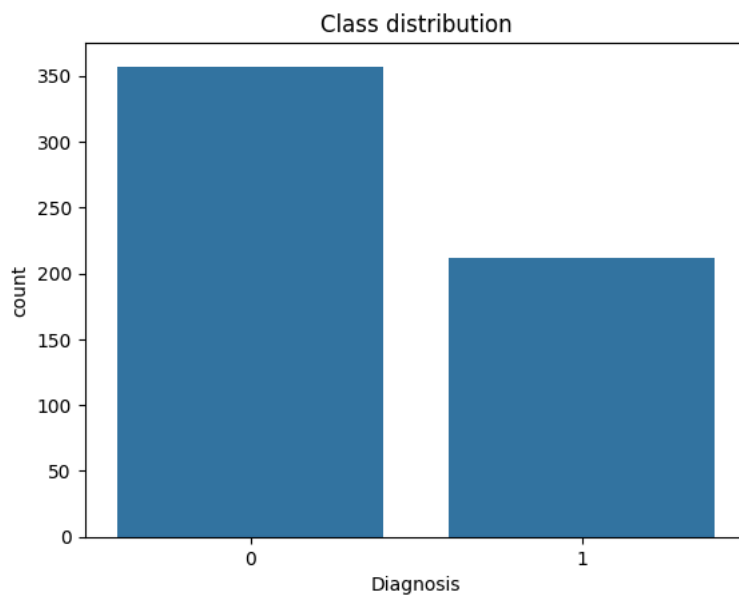
count    symmetry3    fractal_dimension3
mean      0.290076      0.083946
std       0.061867      0.018061
min       0.156500      0.055040
25%       0.250400      0.071460
50%       0.282200      0.080040
75%       0.317900      0.092080
max       0.663800      0.207500

```

[8 rows x 30 columns]

```
#Class distribution

import matplotlib.pyplot as plt
import seaborn as sns
y = y['Diagnosis']
y = y.map({'M':1,'B':0})
sns.countplot(x=y)
plt.title("Class distribution")
plt.show()
```



3. Split the dataset into training and testing sets (80–20)

```
#Train test split

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.2,random_state = 42,stratify = y)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

4. Implement a Bagging classifier using a base estimator.

```
#Bagging classifier

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier

bag_model = BaggingClassifier(estimator = DecisionTreeClassifier(),random_state = 42)
```

5. Implement Boosting classifiers (AdaBoost and/or Gradient Boosting).

```
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier

ada_model = AdaBoostClassifier(random_state=42)

gb_model = GradientBoostingClassifier(random_state=42)
```

6. Construct a Stacked Ensemble using multiple base models.

```
from sklearn.ensemble import StackingClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression

base_models = [
    ('svm', SVC(probability=True)),
    ('nb', GaussianNB()),
```

```

        ('dt', DecisionTreeClassifier())
    ]

    stack_model = StackingClassifier(
        estimators=base_models,
        final_estimator=LogisticRegression(),
        cv=5
    )

```

7. Define hyperparameter search spaces for each ensemble model.

```

bag_params = {
    'n_estimators': [10, 50, 100],
    'max_samples': [0.8, 0.9, 1.0],
    'max_features': [0.8, 1.0]
}

ada_params = {
    'n_estimators': [50, 100, 150],
    'learning_rate': [0.01, 0.05, 0.1]
}

gb_params = {
    'n_estimators': [50, 100],
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 5]
}

```

8. Select hyperparameters using 5-fold cross-validation

```

from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, AdaBoostClassifier, GradientBoostingClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression

# Re-defining models (from AcNHlILE0qmu, IqRGztzI1Iin, e7iliztx1w8o)
bag_model = BaggingClassifier(estimator = DecisionTreeClassifier(), random_state = 42)
ada_model = AdaBoostClassifier(random_state=42)
gb_model = GradientBoostingClassifier(random_state=42)

base_models = [
    ('svm', SVC(probability=True)),
    ('nb', GaussianNB()),
    ('dt', DecisionTreeClassifier())
]
stack_model = StackingClassifier(
    estimators=base_models,
    final_estimator=LogisticRegression(),
    cv=5
)

# Re-defining params (from 9luGnxaj17sr)
bag_params = {
    'n_estimators': [10, 50, 100],
    'max_samples': [0.8, 0.9, 1.0],
    'max_features': [0.8, 1.0]
}
ada_params = {
    'n_estimators': [50, 100, 150],
    'learning_rate': [0.01, 0.05, 0.1]
}
gb_params = {
    'n_estimators': [50, 100],
    'learning_rate': [0.01, 0.1],
    'max_depth': [3, 5]
}

# Bagging
bag_grid = GridSearchCV(bag_model, bag_params, cv=5, scoring=['f1', 'accuracy'], refit='f1', n_jobs=-1)
bag_grid.fit(X_train, y_train)
best_bag = bag_grid.best_estimator_

print("Best Bagging Params:", bag_grid.best_params_)

# AdaBoost
ada_grid = GridSearchCV(ada_model, ada_params, cv=5, scoring=['f1', 'accuracy'], refit='f1', n_jobs=-1)
ada_grid.fit(X_train, y_train)

```

```

best_ada = ada_grid.best_estimator_

print("Best AdaBoost Params:", ada_grid.best_params_)

# Gradient Boosting
gb_grid = GridSearchCV(gb_model, gb_params, cv=5, scoring=['f1', 'accuracy'], refit='f1', n_jobs=-1)
gb_grid.fit(X_train, y_train)
best_gb = gb_grid.best_estimator_

print("Best Gradient Boosting Params:", gb_grid.best_params_)

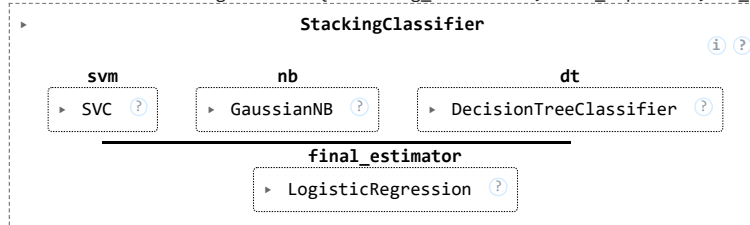
# Stacking (no tuning here, but you can extend)
stack_model.fit(X_train, y_train)

```

```

Best Bagging Params: {'max_features': 0.8, 'max_samples': 0.9, 'n_estimators': 100}
Best AdaBoost Params: {'learning_rate': 0.1, 'n_estimators': 100}
Best Gradient Boosting Params: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 50}

```



9. Evaluate all models using standard performance metrics.

```

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_auc_score

def evaluate(model, X_test, y_test):
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob)

    print("Accuracy:", acc)
    print("Precision:", prec)
    print("Recall:", rec)
    print("F1 Score:", f1)
    print("AUC:", auc)
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

    return acc, prec, rec, f1

print("\nBagging:")
bag_res = evaluate(best_bag, X_test, y_test)

print("\nAdaBoost:")
ada_res = evaluate(best_ada, X_test, y_test)

print("\nGradient Boosting:")
gb_res = evaluate(best_gb, X_test, y_test)

print("\nStacking:")
stack_res = evaluate(stack_model, X_test, y_test)

```

```

Bagging:
Accuracy: 0.956140350877193
Precision: 1.0
Recall: 0.8809523809523809
F1 Score: 0.9367088607594937
AUC: 0.9917328042328043
Confusion Matrix:
[[72  0]
 [ 5 37]]

```

```

AdaBoost:
Accuracy: 0.9649122807017544
Precision: 1.0
Recall: 0.9047619047619048
F1 Score: 0.95
AUC: 0.994378306878307
Confusion Matrix:
[[72  0]
 [ 4 38]]

```

```

Gradient Boosting:
Accuracy: 0.956140350877193
Precision: 0.9743589743589743
Recall: 0.9047619047619048
F1 Score: 0.9382716049382716
AUC: 0.9963624338624338
Confusion Matrix:
[[71  1]
 [ 4 38]]

Stacking:
Accuracy: 0.9649122807017544
Precision: 1.0
Recall: 0.9047619047619048
F1 Score: 0.95
AUC: 0.9970238095238095
Confusion Matrix:
[[72  0]
 [ 4 38]]

```

```

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Function to plot confusion matrix
def plot_confusion_matrix(model, X_test, y_test, title):
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Benign', 'Malignant'])
    disp.plot(cmap=plt.cm.Blues)
    plt.title(title)
    # Use the title for a more descriptive filename
    clean_title = title.replace(' ', '_').replace(':', '') # Basic sanitization for filename
    plt.savefig(fname=f"{clean_title}.png")
    plt.show()

print("Confusion Matrix for Bagging:")
plot_confusion_matrix(best_bag, X_test, y_test, "Bagging Classifier Confusion Matrix")

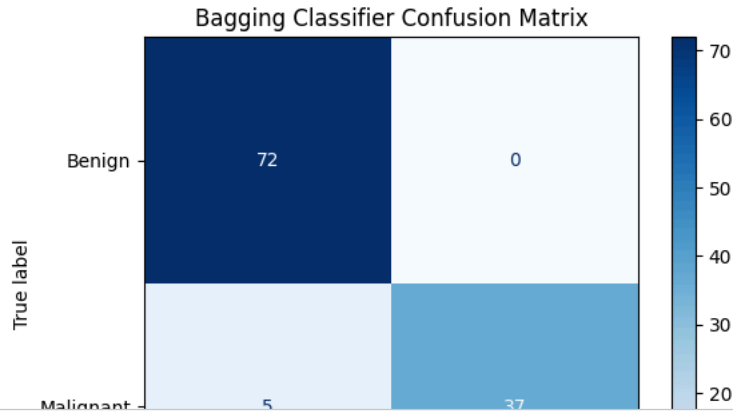
print("Confusion Matrix for AdaBoost:")
plot_confusion_matrix(best_ada, X_test, y_test, "AdaBoost Classifier Confusion Matrix")
# Removed redundant plt.savefig(fname = "Adacm.png")

print("Confusion Matrix for Gradient Boosting:")
plot_confusion_matrix(best_gb, X_test, y_test, "Gradient Boosting Classifier Confusion Matrix")
# Removed redundant plt.savefig(fname="gradcm.png")

print("Confusion Matrix for Stacking:")
plot_confusion_matrix(stack_model, X_test, y_test, "Stacking Classifier Confusion Matrix")
# Removed redundant plt.savefig(fname="stackcm.png")

```

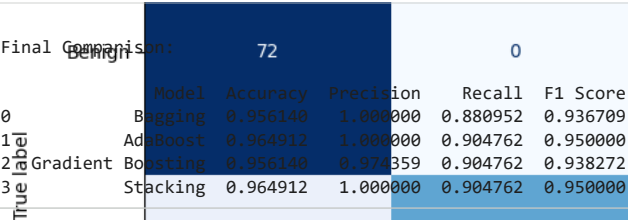

Confusion Matrix for Bagging:



```
import pandas as pd

results = pd.DataFrame({
    "Model": ["Bagging", "AdaBoost", "Gradient Boosting", "Stacking"],
    "Accuracy": [bag_res[0], ada_res[0], gb_res[0], stack_res[0]],
    "Precision": [bag_res[1], ada_res[1], gb_res[1], stack_res[1]],
    "Recall": [bag_res[2], ada_res[2], gb_res[2], stack_res[2]],
    "F1 Score": [bag_res[3], ada_res[3], gb_res[3], stack_res[3]]
})

print("\nFinal Comparison:\n")
print(results)
```



10. Bias-Variance Analysis of Ensemble Models

1. Bagging (Bagging Classifier with Decision Tree):

- **Bias-Variance Behavior:** Bagging primarily aims to reduce variance. Decision trees, as base estimators, are known for their high variance (they can overfit to the training data). By training multiple decision trees on different subsets of the data (bootstrap samples) and averaging their predictions, Bagging effectively reduces this variance, leading to a more stable and generalized model.
- **Observed Performance:** The Bagging model showed good performance with high accuracy and perfect precision. The recall was slightly lower compared to Gradient Boosting and stacking models, suggesting that while it makes accurate positive predictions, it might miss some actual positive cases.

2. Boosting (AdaBoost and Gradient Boosting):

- **Bias-Variance Behavior:** Boosting methods primarily focus on reducing bias. They do this by sequentially building models, where each subsequent model tries to correct the errors of the previous ones. By iteratively improving the model's performance on misclassified instances, boosting reduces the overall bias of the ensemble.
- **Observed Performance (AdaBoost):** AdaBoost achieved excellent performance, matching the best F1-score and accuracy. Its perfect precision and high recall indicate a very effective model in identifying malignant cases with few false positives and false negatives.
- **Observed Performance (Gradient Boosting):** Gradient Boosting also performed very well, with strong accuracy, precision, recall, and F1-score. Similar to AdaBoost, its strong performance suggests a good reduction in bias, allowing it to capture the underlying patterns in the data effectively.

3. Stacking (Stacking Classifier):

- **Bias-Variance Behavior:** Stacking aims to combine the strengths of multiple diverse base models to potentially reduce both bias and variance. By training a meta-learner to make final predictions based on the outputs of the base models, stacking can learn complex relationships and create a more robust model that generalizes better than any single base model.
- **Observed Performance:** The Stacking model performed exceptionally well, achieving the highest F1-score and accuracy along with AdaBoost. Its perfect precision and high recall demonstrate its ability to combine the predictions of its base estimators (SVC, GaussianNB, Decision Tree) for accurate classifications. This indicates a good balance in addressing both bias and variance components.

```
#CM plot
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
```



```
import matplotlib.pyplot as plt

# Function to plot confusion matrix
def plot_confusion_matrix(model, X_test, y_test, title):
    y_pred = model.predict(X_test)
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Benign', 'Malignant'])
    disp.plot(cmap=plt.cm.Blues)
    plt.title(title)
    plt.show()

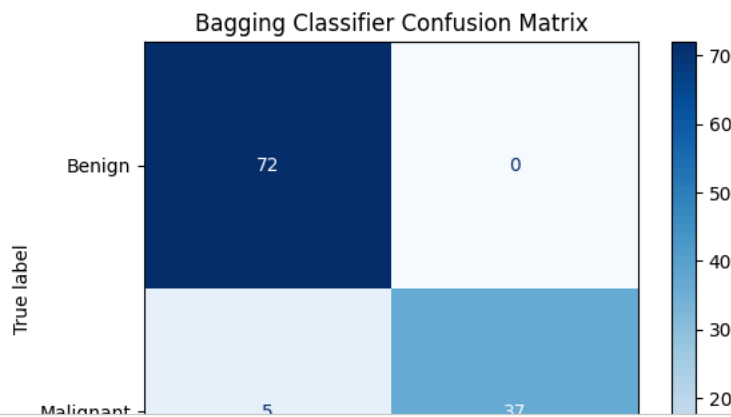
print("Confusion Matrix for Bagging:")
plot_confusion_matrix(best_bag, X_test, y_test, "Bagging Classifier Confusion Matrix")

print("Confusion Matrix for AdaBoost:")
plot_confusion_matrix(best_ada, X_test, y_test, "AdaBoost Classifier Confusion Matrix")

print("Confusion Matrix for Gradient Boosting:")
plot_confusion_matrix(best_gb, X_test, y_test, "Gradient Boosting Classifier Confusion Matrix")

print("Confusion Matrix for Stacking:")
plot_confusion_matrix(stack_model, X_test, y_test, "Stacking Classifier Confusion Matrix")
```


Confusion Matrix for Bagging:



```

from sklearn.metrics import RocCurveDisplay
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))
ax = plt.gca()

# Plot ROC curve for Bagging
RocCurveDisplay.from_estimator(best_bag, X_test, y_test, ax=ax, name='Bagging')

# Plot ROC curve for AdaBoost
RocCurveDisplay.from_estimator(best_ada, X_test, y_test, ax=ax, name='AdaBoost')

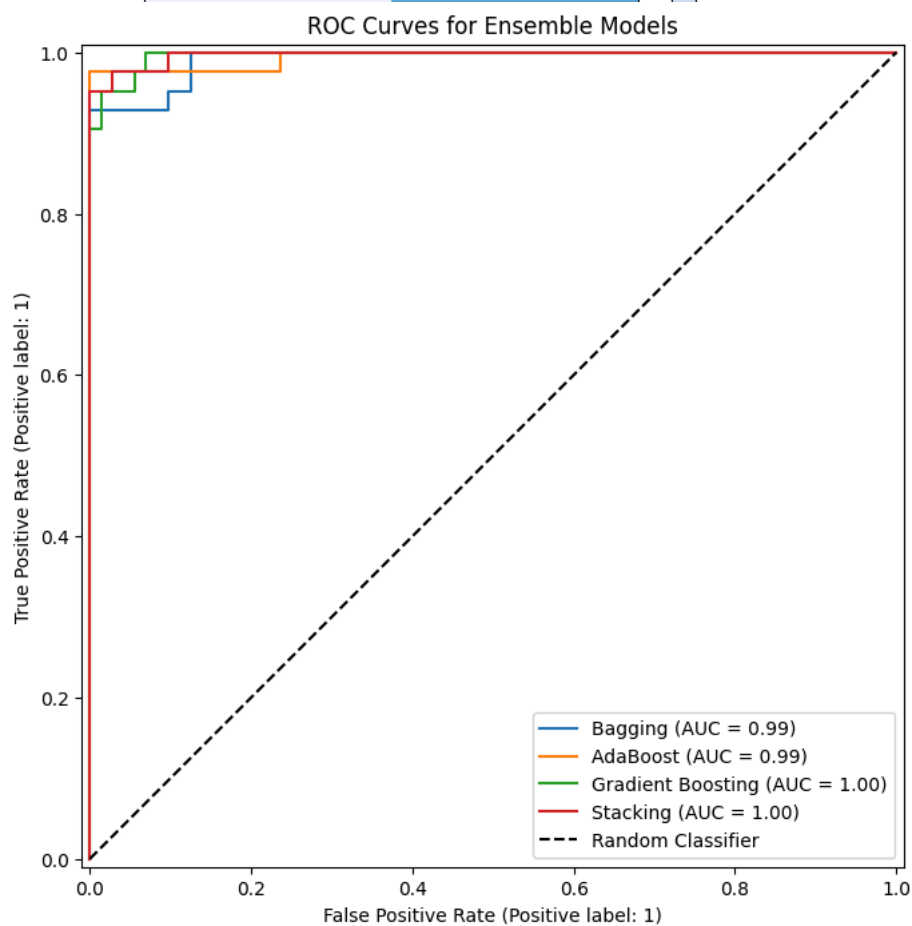
# Plot ROC curve for Gradient Boosting
RocCurveDisplay.from_estimator(best_gb, X_test, y_test, ax=ax, name='Gradient Boosting')

# Plot ROC curve for Stacking
RocCurveDisplay.from_estimator(stack_model, X_test, y_test, ax=ax, name='Stacking')

plt.title('ROC Curves for Ensemble Models')
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier') # Add a diagonal random classifier line
plt.legend()

plt.savefig(fname="roc.png") # Save the plot first
plt.show() # Then show it

```



```

import pandas as pd

# Bagging Results
bagging_best_params = bag_grid.best_params_
bagging_cv_results = bag_grid.cv_results_
bagging_best_index = bag_grid.best_index_
bagging_avg_cv_f1 = bagging_cv_results['mean_test_f1'][bagging_best_index]
bagging_avg_cv_accuracy = bagging_cv_results['mean_test_accuracy'][bagging_best_index]

bagging_data = {
    'n_estimators': [bagging_best_params['n_estimators']],
    'max_samples': [bagging_best_params['max_samples']],
    'max_features': [bagging_best_params['max_features']],
    'Avg CV F1 Score': [bagging_avg_cv_f1],
    'Avg CV Accuracy': [bagging_avg_cv_accuracy]
}
bagging_df = pd.DataFrame(bagging_data)
print("\nTable 1: Bagging Hyperparameter Evaluation (Best Parameters)")
display(bagging_df)

# Display full Bagging CV results for different n_estimators
bagging_results_df = pd.DataFrame(bagging_cv_results)[['param_n_estimators', 'param_max_samples', 'param_max_features', 'mean_test_f1', 'mean_test_accuracy']]
bagging_results_df = bagging_results_df.rename(columns={'param_n_estimators': 'n_estimators', 'param_max_samples': 'max_samples', 'param_max_features': 'max_features'})
print("\nTable 1.1: Bagging Hyperparameter Evaluation (All n_estimators)")
display(bagging_results_df.sort_values(by=['Avg CV F1 Score'], ascending=False))

# Boosting Results (AdaBoost)
adaboost_best_params = ada_grid.best_params_
adaboost_cv_results = ada_grid.cv_results_
adaboost_best_index = ada_grid.best_index_
adaboost_avg_cv_f1 = adaboost_cv_results['mean_test_f1'][adaboost_best_index]
adaboost_avg_cv_accuracy = adaboost_cv_results['mean_test_accuracy'][adaboost_best_index]

adaboost_data = {
    'n_estimators': [adaboost_best_params['n_estimators']],
    'learning_rate': [adaboost_best_params['learning_rate']],
    'Avg CV F1 Score': [adaboost_avg_cv_f1],
    'Avg CV Accuracy': [adaboost_avg_cv_accuracy]
}
adaboost_df = pd.DataFrame(adaboost_data)
print("\nTable 2 (AdaBoost): Boosting Hyperparameter Evaluation (Best Parameters)")
display(adaboost_df)

# Display full AdaBoost CV results for different n_estimators
adaboost_results_df = pd.DataFrame(adaboost_cv_results)[['param_n_estimators', 'param_learning_rate', 'mean_test_f1', 'mean_test_accuracy']]
adaboost_results_df = adaboost_results_df.rename(columns={'param_n_estimators': 'n_estimators', 'param_learning_rate': 'learning_rate'})
print("\nTable 2.1 (AdaBoost): Boosting Hyperparameter Evaluation (All n_estimators)")
display(adaboost_results_df.sort_values(by=['Avg CV F1 Score'], ascending=False))

# Boosting Results (Gradient Boosting)
gradient_boosting_best_params = gb_grid.best_params_
gradient_boosting_cv_results = gb_grid.cv_results_
gradient_boosting_best_index = gb_grid.best_index_
gradient_boosting_avg_cv_f1 = gradient_boosting_cv_results['mean_test_f1'][gradient_boosting_best_index]
gradient_boosting_avg_cv_accuracy = gradient_boosting_cv_results['mean_test_accuracy'][gradient_boosting_best_index]

gradient_boosting_data = {
    'n_estimators': [gradient_boosting_best_params['n_estimators']],
    'learning_rate': [gradient_boosting_best_params['learning_rate']],
    'max_depth': [gradient_boosting_best_params['max_depth']],
    'Avg CV F1 Score': [gradient_boosting_avg_cv_f1],
    'Avg CV Accuracy': [gradient_boosting_avg_cv_accuracy]
}
gradient_boosting_df = pd.DataFrame(gradient_boosting_data)
print("\nTable 2 (Gradient Boosting): Boosting Hyperparameter Evaluation (Best Parameters)")
display(gradient_boosting_df)

# Display full Gradient Boosting CV results for different n_estimators
gradient_boosting_results_df = pd.DataFrame(gradient_boosting_cv_results)[['param_n_estimators', 'param_learning_rate', 'param_max_depth', 'mean_test_f1', 'mean_test_accuracy']]
gradient_boosting_results_df = gradient_boosting_results_df.rename(columns={'param_n_estimators': 'n_estimators', 'param_learning_rate': 'learning_rate', 'param_max_depth': 'max_depth'})
print("\nTable 2.2 (Gradient Boosting): Boosting Hyperparameter Evaluation (All n_estimators)")
display(gradient_boosting_results_df.sort_values(by=['Avg CV F1 Score'], ascending=False))

# Stacked Ensemble Results
# Extract base models and meta-learner names for clarity
base_model_names = [name for name, _ in stack_model.estimators_]
meta_learner_name = type(stack_model.final_estimator).__name__

# stack_res contains the test set performance for the stacked model
stacked_accuracy = stack_res[0]

```

```
stacked_f1 = stack_res[3]

stacked_data = {
    'Base Models': [' ', '.join(base_model_names)],
    'Meta Learner': [meta_learner_name],
    'Test Set Accuracy (%)': [f'{stacked_accuracy:.4f}' ],
    'Test Set F1 Score': [f'{stacked_f1:.4f}']
}
stacked_df = pd.DataFrame(stacked_data)
print("\nTable 3: Stacked Ensemble Evaluation (Test Set Performance)")
display(stacked_df)
```

